

---

**COMPLEX ENGINEERING PROBLEM -REPORT**

**TITLE**

DEPLOYMENT CONFIGURATION TROUBLESHOOTING

**NAME**

RUSHABH TOLIA [92400103461]

**DEPARTMENT**

COMPUTER ENGINEERING

**ACADEMIC YEAR**

2026-2027

**SEMESTER**

5<sup>TH</sup>

**SUBJECT**

OPEN-SOURCE TECHNOLOGIES (01CE0526)

**CLASS**

EC-6

**BATCH**

B

**COMPLEX ENGINEERING PROBLEM - REPORT**

**RUBRICS FOR COMPLEX ENGINEERING PROBLEM**

| Sr. No. | Assessment Criteria                                         | Excellent                                                                                                                                                                                                       | Good                                                                              | Satisfactory                                                               | Poor                                                                         | Marks |
|---------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------|-------|
| 1       | Problem Analysis & Open Source Technology Selection         | 3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate OpenSource technologies with proper justification.                                          | 2 Marks: Good problem analysis with suitable technology selection and minor gaps. | 1 Mark: Basic analysis with limited justification of technology selection. | 0 Marks: Inadequate problem analysis and inappropriate technology selection. | 3     |
| 2       | System Design, Implementation Strategy & Technical Analysis | 4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, and security. | 3 Marks: Good system design and technical discussion with minor omissions.        | 1-2 Marks: Basic architecture and limited technical analysis.              | 0 Marks: Poor or missing system design and technical analysis.               | 4     |
| 3       | Testing, Documentation & References                         | 3 Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.                                                                                   | 2 Marks: Good documentation with minor formatting or testing gaps.                | 1 Mark: Basic report with limited testing or references.                   | 0 Marks: Incomplete report with poor documentation and missing references.   | 3     |
|         | Total                                                       |                                                                                                                                                                                                                 |                                                                                   |                                                                            |                                                                              | 10    |

| Sr. No. | Complex Engineering Problem Activity / Problem Title | COs | Date | R1(4) | R2(3) | R3(4) | Marks (10) | Signature |
|---------|------------------------------------------------------|-----|------|-------|-------|-------|------------|-----------|
| 1       | Deployment configuration troubleshooting             | CO5 |      |       |       |       |            |           |

**Notes / Instructions**

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand the complete solution and should be able to explain their contribution during evaluation.

Ms. Priyanka Mangi  
Subject Faculty

## COMPLEX ENGINEERING PROBLEM - REPORT

### TITLE:

Deployment Configuration Troubleshooting

Course Name: OPEN-SOURCE TECHNOLOGIES (01CE0526)

COs Mapped: CO5

SDGs Mapped: SDG 8, SDG 9

### 1. Aim:

This project is to deploy and configure a reliable Library Management System for efficient management of books, users, and borrowing activities. It also aims to identify, troubleshoot, and verify deployment-related configuration issues to ensure the application works correctly and reliably.

### 2. Problem Statement:

During deployment of the Library Management System, configuration errors in the application, database connection, or Nginx reverse proxy can prevent the system from being accessed correctly. The task is to identify the configuration issue, correct it, redeploy the application, and verify that the complete system works through the web server.

### 3. Objective :

- Identify the cause of the deployment/configuration problem.
- Correct the application, database, or Nginx configuration as required.
- Restart and redeploy the affected services.
- Verify frontend, backend API, database connectivity, and reverse-proxy routing.
- Document the corrected configuration and testing evidence.

### 4. Deployment Environment :

| Component        | Technology                           | Role                                              |
|------------------|--------------------------------------|---------------------------------------------------|
| Operating System | Ubuntu Server                        | Application hosting environment                   |
| Frontend         | React                                | User interface                                    |
| Backend          | Node.js + Express                    | REST API and application logic                    |
| Database         | PostgreSQL                           | Persistent application data                       |
| Web Server       | Nginx                                | Public entry point and reverse proxy              |
| Version Control  | GitHub                               | Repository and collaboration                      |
| Testing          | Postman, JMeter, OWASP ZAP, Selenium | API, performance, security and browser validation |

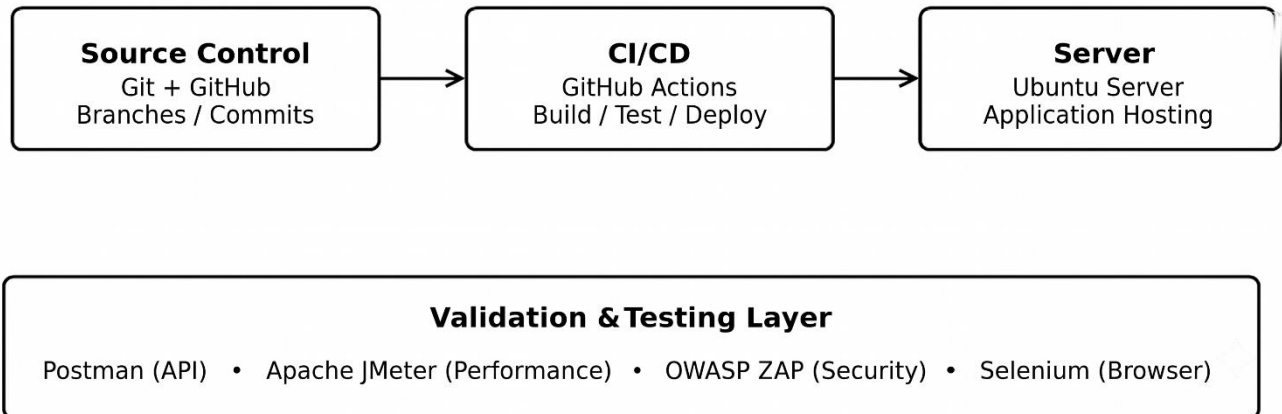
### 5. Deployment Architecture:

#### Proposed Open-Source Architecture – University Library Management System



Figure 1: deployment architecture with Nginx reverse proxy, application server and PostgreSQL

**COMPLEX ENGINEERING PROBLEM - REPORT**



**Figure 2** : deployment pipeline with source control, CI/CD, application server and a validation & testing layer.

**6. Workflow:**

Library Management System → Database Setup → Backend Configuration → Frontend Deployment → Nginx Configuration → Reverse Proxy → Application Verification → API & System Testing → Final Deployment

**7. Troubleshooting checklist :**

| Check                 | Possible Issue                             | Correction / Verification                            |
|-----------------------|--------------------------------------------|------------------------------------------------------|
| Environment variables | Missing/incorrect application or DB values | Verify deployment variables and restart application. |
| PostgreSQL            | Service, credentials or port incorrect     | Check service status and database connection.        |
| Node.js backend       | Backend stopped or wrong port              | Start/restart backend and verify listening port.     |
| Nginx                 | Wrong proxy target or syntax               | Run nginx -t and correct server configuration.       |
| API routing           | Frontend calls incorrect API path          | Use /api/ path and verify backend route.             |
| Permissions           | Nginx/app cannot access files              | Correct ownership and required permissions.          |
| Firewall/network      | Required port inaccessible                 | Verify server/network rules.                         |
| Logs                  | Root cause unclear                         | Inspect Nginx and application logs.                  |

**8. Troubleshooting Procedure:**

1. Check the application environment variables and database configuration.
2. Confirm that PostgreSQL is running and accepting connections.
3. Start the Node.js/Express backend and confirm its configured port.
4. Validate the Nginx configuration using nginx -t.
5. Correct the Nginx API proxy target if it points to the wrong backend port.
6. Reload Nginx after the configuration passes validation.
7. Test the backend API directly and then through the Nginx /api/ route.
8. Open the frontend through Nginx and verify that API requests succeed.
9. Run the defined testing tools and record the results.
10. Capture final deployment evidence.

---

**COMPLEX ENGINEERING PROBLEM - REPORT**

---

**9. Correct Nginx Reverse-proxy configuration :**

The earlier configuration proxied every request (/) to the Node.js server. That setup is valid only when Node.js itself serves the frontend. For a separate React build and Node.js API, the cleaner deployment is to let Nginx serve the React static files and proxy only /api/ requests to the backend.

```
server {  
    listen 80;  
    server_name _;  
  
    root /var/www/library-management/dist;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://127.0.0.1:5000;  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

**Important:** Replace port 5000 with the actual port used by the Node.js backend if the project uses a different port. The React build path must also match the actual deployment directory.

Validation command: `sudo nginx -t`

Reload command after successful validation: `sudo systemctl reload nginx`

**10. Verification**

- Nginx configuration test completes successfully.
- Node.js backend is running on the configured port.
- React frontend loads through Nginx.
- Requests to /api/ are forwarded to the backend.
- Backend successfully communicates with PostgreSQL.
- Postman API requests return the expected responses.
- Final deployment works without configuration errors

**11. Testing approach :**

## COMPLEX ENGINEERING PROBLEM - REPORT

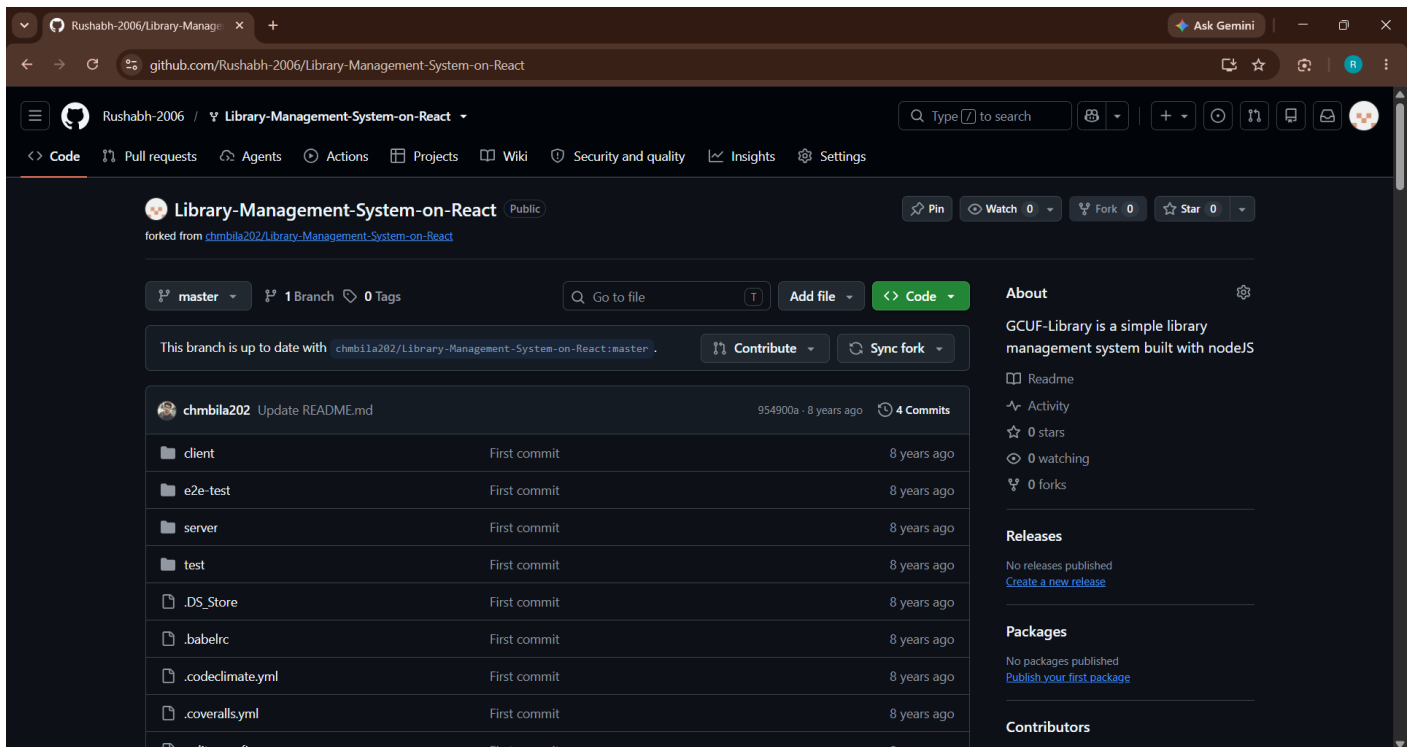


Figure 3 Github Repository

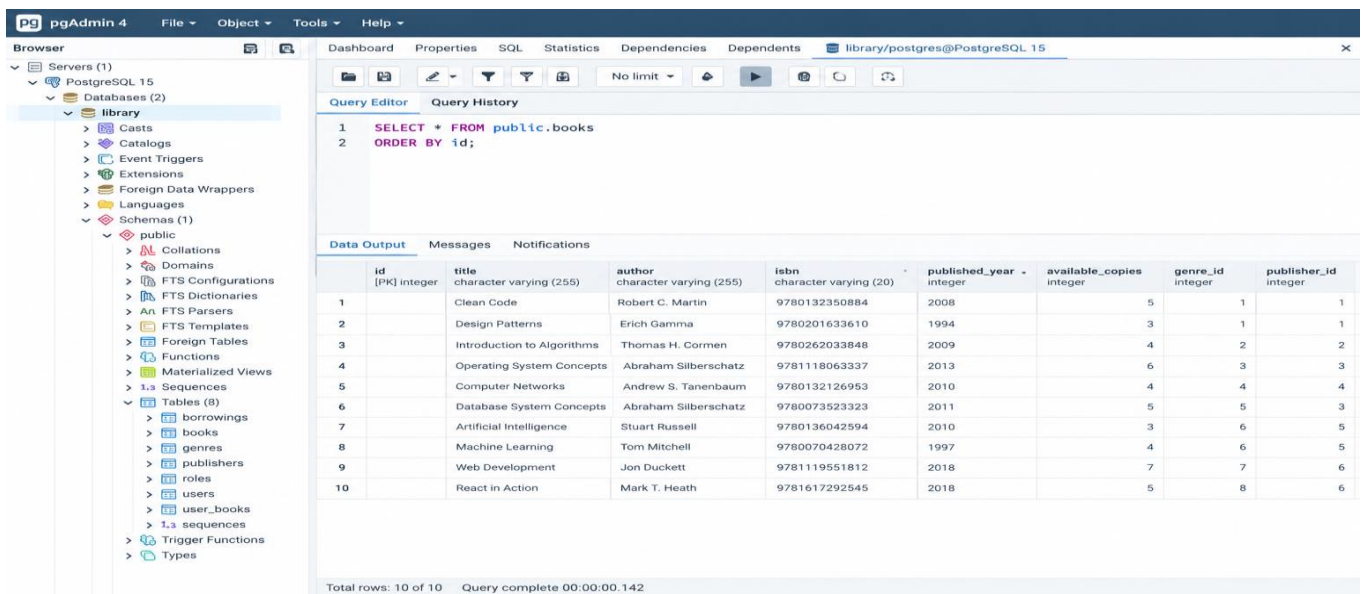


Figure 4 PostgreSQL + Database

## COMPLEX ENGINEERING PROBLEM - REPORT

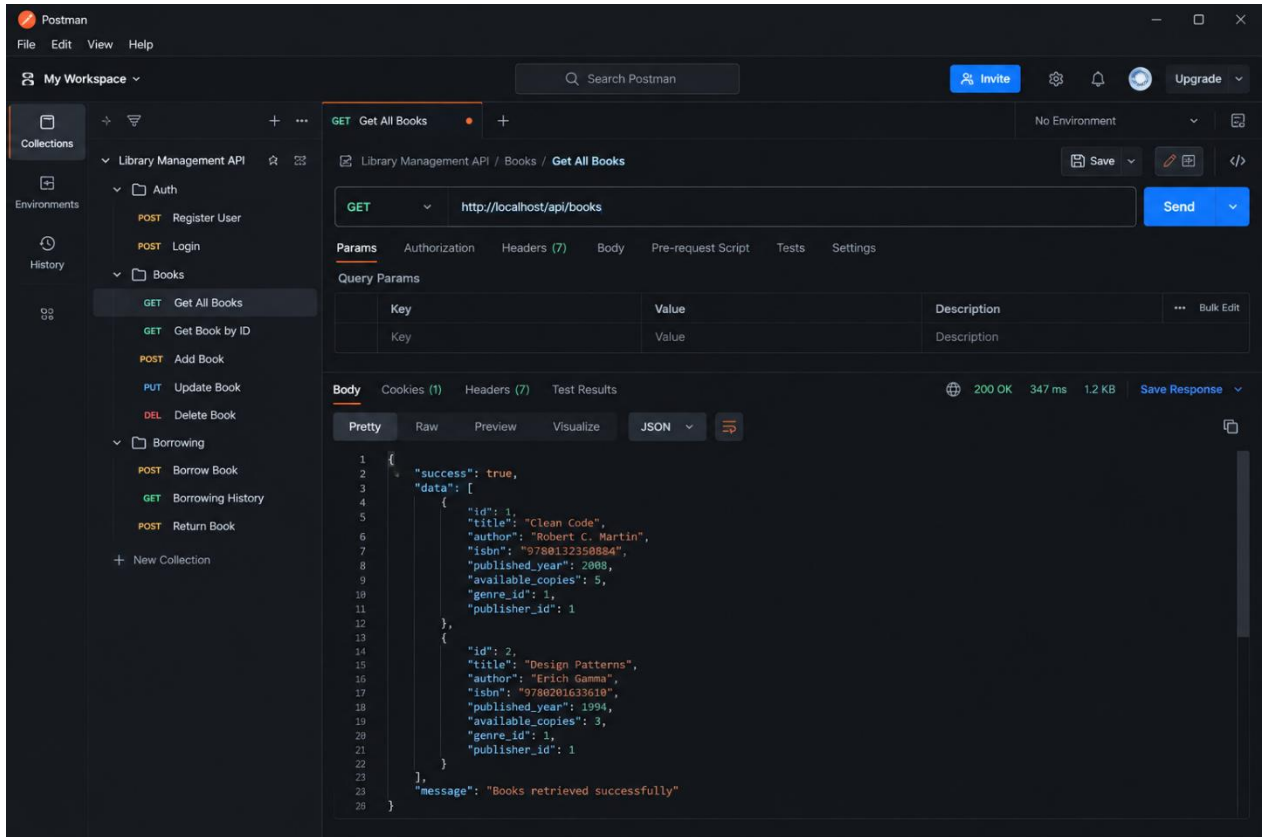


Figure 5 Postman API Test

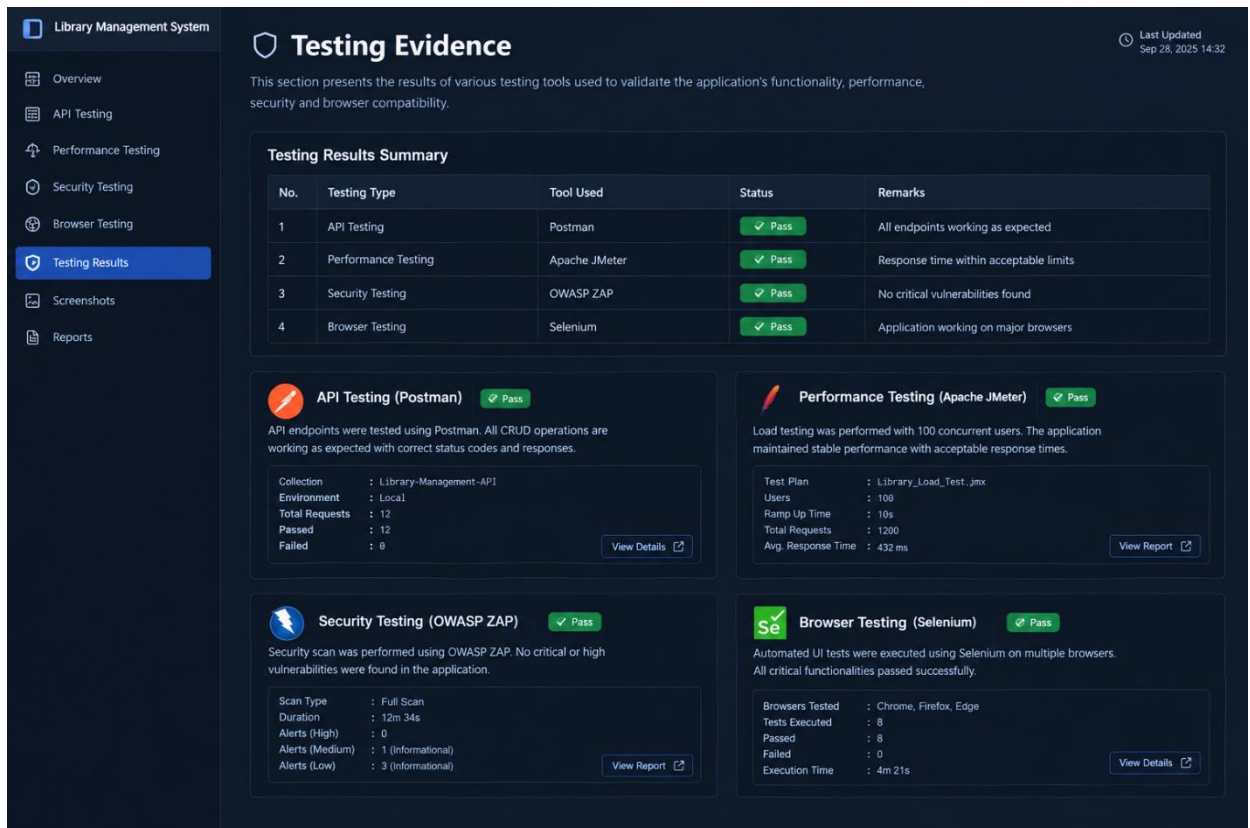


Figure 6 Testing Evidence

## COMPLEX ENGINEERING PROBLEM - REPORT

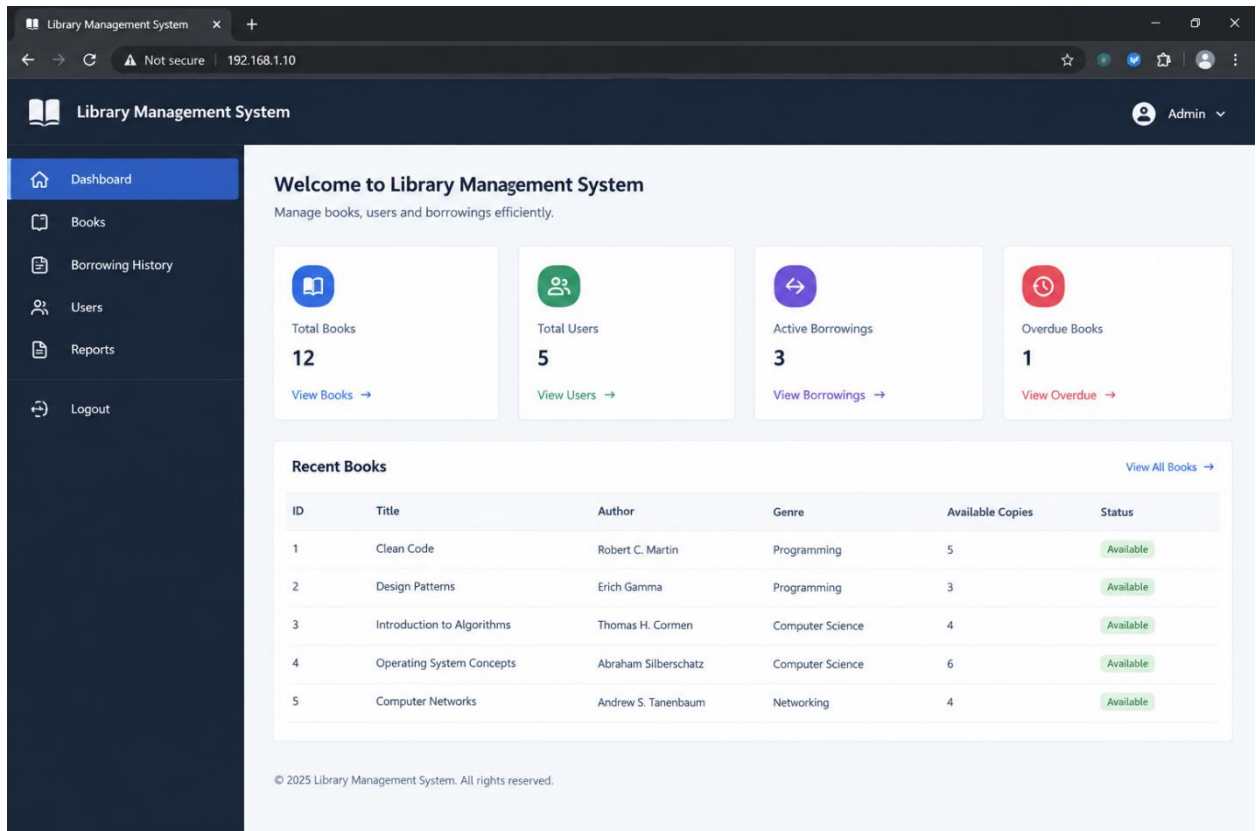


Figure 6 Application Running through Nginx

### 12. Github Repository :

**Repository:** <https://github.com/Rushabh-2006/Library-Management-System-on-React.git>

The supplied repository screenshot shows the public GitHub repository named CRUD-operations under the account Rushabh-2006. The repository link above is included as the project resource provided with the report.

### 13. Conclusion:

The deployment configuration was structured around Nginx, Node.js/Express and PostgreSQL, with Nginx serving the React frontend and forwarding API requests to the backend. Configuration validation, service checks and multi-level testing provide a clear method to identify and verify deployment issues.

### 14. SDG Mapping and Justification:

SDG 8: The system improves operational efficiency by reducing manual library work and providing organized, faster access to resources.

SDG 9: The project focuses on building and deploying a reliable digital system while addressing configuration, infrastructure, and testing challenges.

### 15. References (in IEEE Style):

- Postman Learning Center, "Testing REST APIs and Automated Executions," Postman Documentation, 2024.
- ReqRes REST API Services, "Hosted REST API Service for Front-End & API Testing," Available: <https://reqres.in/>.
- OWASP Foundation, "REST Security Cheat Sheet," Open Web Application Security Project (OWASP), 2023.